Wyjątki i klasy abstrakcyjne

Hierarchia wyjątków

Mechanizm wyłapywania wyjątków bazuje na dziedziczeniu. Np. hierarchia standardowych wyjątków (Exception hierarchy — Built-in Exceptions — Python documentation) jest właśnie hierarchią klas — nazwa wyjątku to nazwa klasy. Klauzula `except` wyłapuje wyjątki wymienionych w niej klas i ich klas pochodnych.

In [2]:
```
try:
    raise IndexError()
except LookupError:
    print("Błąd!")
```

```
Błąd!
```

Definiowanie swoich własnych typów wyjątków polega po prostu na definiowanu odpowiednich klas. Takie klasy powinny dziedziczyć bezpośrednio lub pośrednio po klasie `Exception`. Często cała definicja klasy wyjątku będzie wyglądała tak:

In [3]:
```
class BladUjemnejWartosci(ValueError):
    pass
```

Najważniejsza informacja związana z wyjątkiem, to zazwyczaj po prostu typ wyjątku.

In [4]:
```
def predkosc(droga, czas):
    if droga < 0:
        raise BladUjemnejWartosci("ujemna droga")
    if czas < 0:
        raise BladUjemnejWartosci("ujemny czas")
    if czas == 0:
        raise ValueError("czas musi być dodatni")
    return droga / czas
```

Możemy jej teraz użyć tak:

In [5]:
```
try:
    droga = float(input("droga: "))
    czas = float(input("czas: "))
    print(predkosc(droga, czas))
except ValueError as e:
    print("Błędne wartości!")
```

```
droga: -10
czas: 2
Błędne wartości!
```

Widzimy, że wpisanie ujemnych wartości powoduje wyjątek klasy `BladUjemnejWartosci`. Jest on wyłapywany, ponieważ jest to podklasa klasy `ValueError`.

Błąd `ValueError` może być tutaj spowodowany również wpisaniem przez użytkownika napisu, którego nie da się zinterpretować jako wartości typu `float` (wypróbuj to). Powyższy sposób obsługi wyjątków nie rozróżnia tych sytuacji. Poniższy sposób rozróżnia:

In [6]:
```
try:
    droga = float(input("droga: "))
    czas = float(input("czas: "))
    print(predkosc(droga, czas))
except BladUjemnejWartosci as e:
    print(f"Błędna wartość: {e}")
except ValueError:
    print("Wpisane wartości nie są liczbami.")
```

```
droga: abc
Wpisane wartości nie są liczbami.
```

Odwrócenie kolejności klauzul `except` w powyższym przykładzie jest błędem. Efektem byłoby to, że druga z nich nie zostałaby nigdy użyta — wyjątek `BladUjemnejWartosci` zostałby wyłapany przez pierwszą z nich.

In [7]:
```
try:
    droga = float(input("droga: "))
    czas = float(input("czas: "))
    print(predkosc(droga, czas))
except ValueError:
    print("Wpisane wartości nie są liczbami.")
except BladUjemnejWartosci as e:
    print(f"Błędna wartość: {e}")
```

```
droga: -10
czas: 2
Wpisane wartości nie są liczbami.
```

Klasy abstrakcyjne

Wróćmy do przykładu z klasami `Kolo` i `Kwadrat` oraz funkcją `wyswietl_figure`. Chcemy stworzyć hierarchię klas dla figur podobną do hierarchii dla osób:

```
Figura
├── Kolo
└── Kwadrat
```

Wtedy w funkcji `wyswietl_figure` będzie można sprawdzić za pomocą funkcji `isinstance`, czy argument jest figurą. Zdefiniujmy odpowiednie trzy klasy.

Metod `obwod` i `pole` nie da się sensownie zaimplementować w klasie `Figura`. Moglibyśmy w związku z tym pominąć ich implementację lub wyrzucić w nich odpowiedni wyjątek (`NotImplementedError`) i zaimplementować je w podklasach.

In [8]:
```
import math


class Figura:
    def obwod(self):
        raise NotImplementedError("brak implementacji metody obwod")

    def pole(self):
        raise NotImplementedError("brak implementacji metody pole")


class Kolo(Figura):
    def __init__(self, promien):
        self.promien = promien

    def obwod(self):
        return 2 * math.pi * self.promien

    def pole(self):
        return math.pi * self.promien**2


class Kwadrat(Figura):
    def __init__(self, bok):
        self.bok = bok

    def obwod(self):
        return 4 * self.bok

    def pole(self):
        return self.bok**2


def wyswietl_figure(figura):
    if not isinstance(figura, Figura):
        raise TypeError("potrzebny obiekt klasy Figura")
    print(f"obwód: {figura.obwod()}")
    print(f"pole: {figura.pole()}")
```

In [9]:
```
kwadrat = Kwadrat(5)
kolo = Kolo(3)
```

Obiekty `kwadrat` i `kolo` są egzemplarzami klas pochodnych od `Figura`, więc spełniają również sprawdzenie `isinstance(..., Figura)`.

In [10]:
```
wyswietl_figure(kwadrat)
```

```
obwód: 20
pole: 25
```

In [11]:
```python
wyswietl_figure(kolo)
```

```
obwód: 18.84955592153876
pole: 28.274333882308138
```

Zdefiniujmy klasę z metodami `pole` i `obwod`, ale niebędącą podklasą klasy `Figura`.

In [12]:
```python
class A:
    def pole(self):
        return 3

    def obwod(self):
        return 5
```

In [13]:
```python
a = A()
```

In [14]:
```python
a.obwod()
```

Out[14]:
```
5
```

In [15]:
```python
a.pole()
```

Out[15]:
```
3
```

Obiekt `a` ma wprawdzie metody `pole` i `obwod`, ale jego klasa `A` nie dziedziczy po `Figura`. Dlatego sprawdzenie w funkcji `wyswietl_figure` odrzuci go, zgłaszając `TypeError`:

In [16]:
```python
wyswietl_figure(a)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[16], line 1
----> 1 wyswietl_figure(a)

Cell In[8], line 36, in wyswietl_figure(figura)
     34 def wyswietl_figure(figura):
     35     if not isinstance(figura, Figura):
---> 36         raise TypeError("potrzebny obiekt klasy Figura")
     37     print(f"obwód: {figura.obwod()}")
     38     print(f"pole: {figura.pole()}")

TypeError: potrzebny obiekt klasy Figura
```

Problemem jest teraz to, że możemy utworzyć obiekty typu `Figura`, co nie ma sensu, bo nie możemy nic z takim obiektem zrobić — nie ma sensu liczenie pola lub obwodu jeżeli nie znamy konkretnego typu figury.

In [17]:
```
figura = Figura()
```

Obiekt `figura` można jeszcze utworzyć, ale próba obliczenia jego obwodu zakończy się `NotImplementedError`. Metoda klasy `Figura` nie zawiera obliczenia, tylko zgłasza ten wyjątek:

In [18]:
```
wyswietl_figure(figura)
```

```
---------------------------------------------------------------------------
NotImplementedError                       Traceback (most recent call last)
Cell In[18], line 1
----> 1 wyswietl_figure(figura)

Cell In[8], line 37, in wyswietl_figure(figura)
     34 def wyswietl_figure(figura):
     35     if not isinstance(figura, Figura):
     36         raise TypeError("potrzebny obiekt klasy Figura")
---> 37     print(f"obwód: {figura.obwod()}")
     38     print(f"pole: {figura.pole()}")

Cell In[8], line 6, in Figura.obwod(self)
      5     def obwod(self):
----> 6         raise NotImplementedError("brak implementacji metody obwod")

NotImplementedError: brak implementacji metody obwod
```

Klasa `Figura` jest nam potrzebna tylko po to, żeby móc utworzyć hierarchię klas, ale każda figura powinna mieć jakiś **konkretny** typ — `Kwadrat` lub `Kolo`. Nie powinno być obiektów typu `Figura`. Żeby policzyć pole lub obwód musimy znać ten konkretny typ — każdy klasa implementująca konkretną figurę dostarcza odpowiednią implementację.

Metody takie jak `pole` i `obwod` nazywa się metodami **abstrakcyjnymi** — każda figura ma pole i obwód (każda instancja klasy `Figura` będzie miała odpowiednie metody), ale nie wiemy jak je policzyć dopóki nie znamy konkretnego typu figury.

Klasę taką jak `Figura` nazywa się **klasą abstrakcyjną**. Klasy takie jak `Kolo` i `Kwadrat`, to tzw. **klasy konkretne**.

Jeżeli chcemy wprost wyrazić to, że klasa jest abstrakcyjna, to możemy użyć modułu `abc` (skrót od ang. *abstract base classes* — abstrakcyjne klasy bazowe). Klasa abstrakcyjna powinna dziedziczyć po klasie `abc.ABC`, a metody abstrakcyjne powinny być oznaczone dekoratorem `abc.abstractmethod`.

In [19]:
```
import abc
import math


class Figura(abc.ABC):
```

```
    @abc.abstractmethod
    def obwod(self):
        pass

    @abc.abstractmethod
    def pole(self):
        pass


class Kolo(Figura):
    def __init__(self, promien):
        self.promien = promien

    def obwod(self):
        return 2 * math.pi * self.promien

    def pole(self):
        return math.pi * self.promien**2


class Kwadrat(Figura):
    def __init__(self, bok):
        self.bok = bok

    def obwod(self):
        return 4 * self.bok

    def pole(self):
        return self.bok**2


def wyswietl_figure(figura):
    if not isinstance(figura, Figura):
        raise TypeError("potrzebny obiekt klasy Figura")
    print(f"obwód: {figura.obwod()}")
    print(f"pole: {figura.pole()}")
```

In [20]:
```
kwadrat = Kwadrat(5)
```

In [21]:
```
wyswietl_figure(kwadrat)
```

```
obwód: 20
pole: 25
```

Teraz błąd jest zgłaszany już przy próbie utworzenia obiektu klasy abstrakcyjnej, a nie dopiero przy próbie wywołania metody.

Nie można już utworzyć obiektu samej klasy `Figura`: jej metody `pole` i `obwod` są abstrakcyjne. Poniższa próba zakończy się wyjątkiem `TypeError`:

In [22]:
```
f = Figura()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 1
----> 1 f = Figura()

TypeError: Can't instantiate abstract class Figura without an implementation
for abstract methods 'obwod', 'pole'
```

Jeżeli podklasa klasy abstrakcyjnej nie dostarczy implementacji wszystkich metod abstrakcyjnych, to sama będzie klasą abstrakcyjną.

In [23]:
```
class Rownoleglobok(Figura):
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def obwod(self):
        return 2 * (self.a + self.b)
```

Klasa `Rownoleglobok` definiuje `obwod`, ale nie implementuje abstrakcyjnej metody `pole`. Nadal jest więc klasą abstrakcyjną i próba utworzenia jej obiektu spowoduje `TypeError`:

In [24]:
```
rownoleglobok = Rownoleglobok(3, 4)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[24], line 1
----> 1 rownoleglobok = Rownoleglobok(3, 4)

TypeError: Can't instantiate abstract class Rownoleglobok without an impleme
ntation for abstract method 'pole'
```

In [25]:
```
class Prostokat(Rownoleglobok):
    def __init__(self, a, b):
        super().__init__(a, b)

    def pole(self):
        return self.a * self.b
```

In [26]:
```
prostokat = Prostokat(3, 4)
```

In [27]:
```
prostokat.obwod()
```

Out[27]: 14

In [28]:
```
prostokat.pole()
```

Out[28]: 12